OpenAPI Pagination Extension
Specification

Version 0.1.0 — Draft Proposal

An Extension to the OpenAPI Specification 3.x

Proposed for inclusion at: https://spec.openapis.org/

Table of Contents

1. Introduction

1.1 Abstract

This document specifies the Pagination Extension for the OpenAPI Specification (OAS). It defines a new top-level component type, paginationSchemes, which allows API authors to formally describe how pagination is implemented in their APIs. The extension follows the structural and semantic conventions established by the securitySchemes component in OAS 3.x, and integrates at both the component definition level and the operation level.

1.2 Status of This Document

This is a Draft Proposal intended for review by the OpenAPI Initiative Technical Steering Committee. It is not yet a ratified standard. Implementers are encouraged to experiment and provide feedback.

1.3 Motivation

The OpenAPI Specification provides rich mechanisms for describing authentication (securitySchemes), data models (schemas), and communication patterns (operations). However, pagination — a near-universal concern in REST API design — has no first-class representation. API consumers must infer pagination behavior from parameter names, response shapes, and documentation prose.

This lack of standardization has several practical consequences:

- Client SDK generators cannot automatically produce paginated iteration helpers.
- API gateway and proxy tools cannot transparently handle pagination on behalf of callers.
- Documentation tools cannot render pagination behavior in a structured, consistent way.
- Contract testing tools cannot validate pagination invariants automatically.

This extension addresses these gaps by providing a structured vocabulary for pagination description, analogous to how securitySchemes addresses authentication.

1.4 Design Principles

- **Additive only.** This extension adds new fields and component types. It does not modify or deprecate any existing OAS field.
- **Convention-friendly.** Schemes may be applied to operations explicitly or inferred automatically from parameter presence, minimising annotation burden for well-structured APIs.
- **Scheme-agnostic.** The extension supports all common pagination strategies — page-number, cursor/token-based, and link-header — and is open to extension for novel strategies.
- **Composable.** An operation may support multiple pagination schemes simultaneously.
- **Tooling-ready.** Field semantics are precise enough for code generators, validators, and documentation renderers to act on them without heuristics.

2. Terminology

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in RFC 2119.

The following terms are used throughout this specification:

Field Name	Type	Required	Description
`Pagination Scheme`	`concept`	—	A named, reusable definition of how a single pagination strategy is implemented in terms of request inputs and response outputs.
`Scheme Type`	`concept`	—	One of the enumerated strategies: pageNumber, pageToken, or nextLink.
`Auto-detection`	`concept`	—	The process by which a tool infers that a pagination scheme applies to an operation based on parameter name matching, without explicit annotation.
`Cursor`	`concept`	—	An opaque value returned by the server that encodes a position in the result set and is supplied by the client on the next request.
`Continuation Link`	`concept`	—	A full URL returned by the server that, when followed, yields the next page of results.

3. Schema Overview

The extension introduces the following additions to the OpenAPI document structure:

- **components/paginationSchemes** — A map of named Pagination Scheme Object definitions (Section 4).
- **operation/pagination** — An optional array of Pagination Application Objects that associates schemes with a specific operation (Section 5).

The overall relationship mirrors securitySchemes and security:

```
openapi: "3.1.0"
info:
  title: Example API
  version: 1.0.0

components:
  paginationSchemes:
    standardPageNumber:     # <-- scheme definition
      type: pageNumber
      ...

paths:
  /items:
    get:
      pagination:           # <-- operation-level application
        - scheme: standardPageNumber
      ...
```

4. Pagination Scheme Object

A Pagination Scheme Object describes a single, self-contained pagination strategy. It is defined once, under components/paginationSchemes, and may be referenced from any number of operations.

4.1 Fixed Fields

Field Name	Type	Required	Description
`type`	`string`	**Yes**	The pagination strategy. One of: pageNumber, pageToken, or nextLink. See Section 4.2.
`description`	`string`	No	A human-readable description of this pagination scheme, supporting CommonMark syntax.
`request`	`Request Pagination Fields Object`	No	Describes the inputs the client provides to navigate to a given page. See Section 4.3.
`response`	`Response Pagination Fields Object`	No	Describes the outputs the server returns to enable navigation to subsequent pages. See Section 4.4.
`autoDetect`	`boolean | Auto-Detect Object`	No	Controls automatic association of this scheme with operations. Defaults to true. See Section 6.

NOTE: A Pagination Scheme Object MUST contain at least one of request or response. A scheme that defines neither provides no actionable information to tooling.

4.2 Scheme Types

4.2.1 pageNumber

The client requests a specific page by ordinal number and, optionally, specifies the number of items per page. The server returns items for that page. This is the simplest and most widely used strategy.

Canonical parameters: page (1-indexed ordinal), limit or pageSize or per_page (items per page).

4.2.2 pageToken

The client sends an opaque cursor or token on each request (except the first). The server returns a new token with each response that the client must supply on the next request. The client cannot jump to an arbitrary page.

Canonical parameters: pageToken, cursor, after, before (request); nextPageToken, nextCursor (response).

4.2.3 nextLink

The server returns a complete URL representing the next page. The client simply follows the URL. The URL may encode any internal state (tokens, offsets, etc.); the client treats it as opaque.

Canonical locations: a next field in the response body, or a Link header with rel="next".

4.3 Request Pagination Fields Object

Describes the inputs clients provide to control pagination. All fields are optional; include only those that apply.

Field Name	Type	Required	Description
`queryParameters`	`Map[string, Request Field Object]`	No	Query string parameters that control pagination. Keys are parameter names as they appear in the URL.
`bodyFields`	`Map[string, Request Field Object]`	No	Fields in the request body (e.g., a JSON request body for POST-based search endpoints) that control pagination.
`headerFields`	`Map[string, Request Field Object]`	No	Request headers that control pagination (uncommon but supported).

4.3.1 Request Field Object

Field Name	Type	Required	Description
`description`	`string`	No	Human-readable description of this parameter's role in pagination.
`schema`	`Schema Object | Reference Object`	No	The OAS Schema Object describing the field's type and constraints.
`role`	`string`	No	Semantic role of this field. One of: page, pageSize, pageToken, cursor, offset. See Section 4.5.
`required`	`boolean`	No	Whether this field is required for all paginated requests. Defaults to false.

4.4 Response Pagination Fields Object

Describes the outputs the server returns to support navigation to subsequent pages.

Field Name	Type	Required	Description
`bodyFields`	`Map[string, Response Field Object]`	No	Fields in the response body that carry pagination state or links.
`headers`	`Map[string, Response Field Object]`	No	Response headers that carry pagination state or links (e.g., Link, X-Next-Page).

4.4.1 Response Field Object

Field Name	Type	Required	Description
`description`	`string`	No	Human-readable description of this field's role.
`schema`	`Schema Object | Reference Object`	No	The OAS Schema Object describing the field's type.
`role`	`string`	No	Semantic role. One of: nextPageToken, nextCursor, nextLink, totalCount, totalPages, pageSize, currentPage. See Section 4.5.

4.5 Semantic Roles

Roles provide machine-readable semantics for individual fields, enabling generic client code to locate the relevant value without prior knowledge of the specific field name.

Role Value	Direction	Semantics
`page`	Request	The 1-indexed (or 0-indexed, as noted) page number requested by the client.
`pageSize`	Request / Response	The maximum number of items per page. May appear in both request (as a hint) and response (as confirmation).
`offset`	Request	A zero-based byte or item offset into the full result set.
`pageToken`	Request	An opaque token returned by the server on a previous response, identifying the position from which to resume.
`cursor`	Request	Synonym for pageToken; used when the cursor is a structural concept in the API (e.g., GraphQL-style connections).
`nextPageToken`	Response	A token the client must send on the next request to retrieve the following page. Absence (or null) signals the final page.
`nextCursor`	Response	Synonym for nextPageToken.
`nextLink`	Response	A fully-qualified URL the client should request to retrieve the following page.
`totalCount`	Response	The total number of items across all pages at the time of the request.
`totalPages`	Response	The total number of pages given the current pageSize.
`currentPage`	Response	The page number of the current response.

5. Operation-Level Pagination

5.1 The pagination Field

An operation MAY include a pagination field at the operation object level. Its value is an array of Pagination Application Objects. Each object associates one pagination scheme with the operation, optionally overriding fields defined in the scheme.

If the pagination field is absent, tools SHOULD apply auto-detection rules (Section 6) to infer applicable schemes.

An empty array ([]) explicitly declares that the operation does not support any pagination scheme and disables auto-detection for that operation.

5.2 Pagination Application Object

Field Name	Type	Required	Description
`scheme`	`string`	**Yes**	The name of a Pagination Scheme Object defined in components/paginationSchemes, or an inline Pagination Scheme Object (not recommended for reusability).
`overrides`	`Partial Pagination Scheme Object`	No	A partial Pagination Scheme Object whose fields are merged (deeply) with the referenced scheme. Useful for renaming parameters or changing roles on a per-operation basis.
`description`	`string`	No	A human-readable annotation describing pagination behaviour specific to this operation.

NOTE: When overrides is provided, fields specified in overrides take precedence over those in the referenced scheme. Fields not mentioned in overrides retain their scheme-level values. This merging applies recursively to nested objects.

6. Auto-Detection

6.1 Purpose

Many APIs are internally consistent: if they support page-number pagination, every paginated endpoint uses the same parameter name (e.g., page). Requiring explicit pagination annotations on every operation would be burdensome. Auto-detection allows a scheme to declare the rules by which it can be inferred from the existing operation definition.

6.2 The autoDetect Field

The autoDetect field on a Pagination Scheme Object controls whether and how auto-detection applies. It accepts three forms:

- **true** (default) — Auto-detection is enabled using the default rules described in Section 6.3.
- **false** — Auto-detection is disabled. This scheme applies only to operations that explicitly reference it.
- **Auto-Detect Object** — Auto-detection is enabled with custom matching rules (see Section 6.4).

6.3 Default Auto-Detection Rules

When autoDetect is true, a scheme is inferred to apply to an operation if and only if ALL of the following conditions hold:

- The operation does not have an explicit pagination field (or has no entry for this scheme).
- For every queryParameter defined in the scheme's request.queryParameters map, the operation defines a query parameter with an identical name.
- For every bodyField defined in the scheme's request.bodyFields map, the operation's request body schema includes a property with an identical name.

Response fields and headers are NOT considered during auto-detection matching. The absence of a matching response field does not prevent a scheme from being inferred.

> **RATIONALE:** Response field names are more variable and often co-exist with other concerns. Matching on request parameters alone is a more reliable signal of intent. If request parameter matching is insufficient, use an explicit Auto-Detect Object or explicit operation annotations.

6.4 Auto-Detect Object

For fine-grained control, the autoDetect field may be set to an Auto-Detect Object:

Field Name	Type	Required	Description
`matchQueryParams`	`boolean`	No	Whether to apply query parameter name matching. Defaults to true.
`matchBodyFields`	`boolean`	No	Whether to apply request body field name matching. Defaults to true.
`matchResponseFields`	`boolean`	No	Whether to also require matching response body fields. Defaults to false.
`matchHeaders`	`boolean`	No	Whether to also require matching response headers. Defaults to false.
`requireAll`	`boolean`	No	If true, ALL configured match dimensions must match. If false, ANY match is sufficient. Defaults to true.

7. Reference Objects

Pagination Scheme Objects may be referenced using the standard OAS Reference Object ($ref) mechanism, consistent with other component types.

```
components:
  paginationSchemes:
    tokenPagination:
      type: pageToken
      ...

paths:
  /search:
    get:
      pagination:
        - scheme: tokenPagination  # reference by name
```

Inline $ref within pagination application objects is not supported. Schemes must be defined in components/paginationSchemes and referenced by their key name.

8. Complete Examples

8.1 Page-Number Pagination

A simple paginated list endpoint using ordinal page numbers and configurable page size.

```
components:
  paginationSchemes:

    pageNumberPagination:
      type: pageNumber
      description: >
        Standard page-number pagination. Use `page` to select a page
        (1-indexed) and `limit` to control the page size (max 100).
      autoDetect: true

      request:
        queryParameters:
          page:
            role: page
            required: false
            schema:
              type: integer
              minimum: 1
              default: 1
            description: The 1-indexed page number to retrieve.
          limit:
            role: pageSize
            required: false
            schema:
              type: integer
              minimum: 1
              maximum: 100
              default: 20
            description: Maximum number of items to return.
```

```yaml
      response:
        bodyFields:
          total:
            role: totalCount
            schema:
              type: integer
            description: Total number of matching items across all pages.
          page:
            role: currentPage
            schema:
              type: integer
          totalPages:
            role: totalPages
            schema:
              type: integer

paths:
  /products:
    get:
      summary: List all products
      # `pagination` omitted — auto-detected via `page` and `limit` query params
      parameters:
        - name: page
          in: query
          schema: { type: integer, default: 1 }
        - name: limit
          in: query
          schema: { type: integer, default: 20, maximum: 100 }
      responses:
        "200":
          content:
            application/json:
              schema:
                type: object
                properties:
                  items:
                    type: array
                    items: { $ref: "#/components/schemas/Product" }
                  total:      { type: integer }
                  page:       { type: integer }
                  totalPages: { type: integer }
```

8.2 Cursor / Page-Token Pagination

A search endpoint that uses opaque continuation tokens and returns the next token in the response body.

```yaml
components:
  paginationSchemes:

    cursorPagination:
      type: pageToken
      description: >
        Opaque cursor-based pagination. Supply the `pageToken` returned
        in the previous response to advance to the next page.
```

```yaml
      autoDetect: true

      request:
        queryParameters:
          pageToken:
            role: pageToken
            required: false
            schema:
              type: string
            description: >
              Opaque token from the previous response's `nextPageToken`
              field. Omit on the first request.
          pageSize:
            role: pageSize
            required: false
            schema:
              type: integer
              minimum: 1
              maximum: 500
              default: 50

      response:
        bodyFields:
          nextPageToken:
            role: nextPageToken
            schema:
              type: string
              nullable: true
            description: >
              Token for the next page. Null or absent when this is the
              final page.

paths:
  /events:
    get:
      summary: Stream events
      parameters:
        - name: pageToken
          in: query
          schema: { type: string }
        - name: pageSize
          in: query
          schema: { type: integer, default: 50 }
      responses:
        "200":
          content:
            application/json:
              schema:
                type: object
                properties:
                  events:
                    type: array
                    items: { $ref: "#/components/schemas/Event" }
                  nextPageToken:
                    type: string
                    nullable: true
```

8.3 Link-Header Pagination (nextLink)

Pagination where the server returns a next URL in a standard HTTP Link response header, as used by GitHub and other REST APIs.

```yaml
components:
  paginationSchemes:

    linkHeaderPagination:
      type: nextLink
      description: >
        RFC 8288 Link-header pagination. The server includes a
        Link header with rel="next" pointing to the next page.
      autoDetect:
        matchQueryParams: false
        matchBodyFields: false
        matchResponseHeaders: true
        requireAll: false

      request:
        queryParameters:
          per_page:
            role: pageSize
            required: false
            schema:
              type: integer
              minimum: 1
              maximum: 100
              default: 30

      response:
        headers:
          Link:
            role: nextLink
            schema:
              type: string
            description: >
              RFC 8288 Link header. May include rel="next", rel="prev",
              rel="first", and rel="last" relations.

paths:
  /repos/{owner}/{repo}/issues:
    get:
      summary: List repository issues
      pagination:
        - scheme: linkHeaderPagination
      parameters:
        - name: owner
          in: path
          required: true
          schema: { type: string }
        - name: repo
          in: path
          required: true
          schema: { type: string }
        - name: per_page
          in: query
          schema: { type: integer, default: 30 }
      responses:
        "200":
          headers:
            Link:
```

```
              schema: { type: string }
          content:
            application/json:
              schema:
                type: array
                items: { $ref: "#/components/schemas/Issue" }
```

8.4 Multiple Schemes on One Operation

An operation may support more than one pagination scheme simultaneously. In this example, the API accepts either a page number or a cursor token, and returns both a next-page token and a next link.

```
paths:
  /search:
    get:
      summary: Search items
      pagination:
        - scheme: pageNumberPagination
          description: Use `page` and `limit` for page-number access.
        - scheme: cursorPagination
          description: Use `pageToken` for forward-only cursor access.
      parameters:
        - name: q
          in: query
          required: true
          schema: { type: string }
        - name: page
          in: query
          schema: { type: integer, default: 1 }
        - name: limit
          in: query
          schema: { type: integer, default: 20 }
        - name: pageToken
          in: query
          schema: { type: string }
```

8.5 Operation-Level Override

An operation that mostly conforms to a shared scheme, but uses a different parameter name for the page token.

```
paths:
  /legacy/items:
    get:
      summary: List items (legacy endpoint)
      pagination:
        - scheme: cursorPagination
          overrides:
            request:
```

```
              queryParameters:
                continuation:     # renamed from pageToken
                  role: pageToken
                  schema:
                    type: string
      parameters:
        - name: continuation
          in: query
          schema: { type: string }
```

9. Conformance

9.1 Document Conformance

An OpenAPI document that uses this extension MUST satisfy the following requirements:

- All schemes referenced in operation pagination arrays MUST be defined in components/paginationSchemes.
- All Pagination Scheme Objects MUST have a valid type field (one of pageNumber, pageToken, nextLink).
- All Pagination Scheme Objects MUST define at least one of request or response.
- Role values in Request Field Objects and Response Field Objects MUST be from the defined vocabulary in Section 4.5, or be an extension value prefixed with x-.

9.2 Tool Conformance

Tools that claim conformance with this extension (e.g., SDK generators, documentation renderers) SHOULD:

- Parse and expose all fields defined in this specification.
- Implement auto-detection logic as described in Section 6.
- Apply operation-level overrides when constructing the effective scheme for an operation.
- Treat unknown role values (not prefixed with x-) as an error or emit a warning.

9.3 Extensibility

All objects in this extension support the standard OAS extension mechanism. Any field prefixed with `x-` is a specification extension and MUST be ignored by conforming tools that do not recognise it.

10. Relationship to the OpenAPI Specification

10.1 Placement in the OAS Document

This extension adds one new key to the Components Object and one new key to the Operation Object. Both are treated as specification extensions (x- prefix not required because this document ratifies them) once adopted. Prior to adoption, implementors SHOULD prefix the fields as follows for forward compatibility:

- `x-paginationSchemes` in place of `paginationSchemes` under `components`
- `x-pagination` in place of `pagination` on operation objects

> **NOTE:** The x- prefix forms are semantically equivalent and will be aliased to the ratified names once this extension is formally adopted into the specification.

10.2 Interaction with parameters

Pagination Scheme Objects describe the semantic meaning of parameters already declared in the operation's parameters array or request body schema. They do not replace or duplicate parameter declarations. A tool SHOULD validate that query parameters named in a scheme's request.queryParameters map are also declared in the operation's parameters.

10.3 Interaction with responses

Similarly, response body fields named in a scheme's response.bodyFields map SHOULD correspond to properties in the operation's response schema. A tool SHOULD emit a warning when a referenced field is absent from the response schema.

10.4 Interaction with links

The OAS Link Object provides a way to express navigational relationships between operations. Pagination schemes complement Link Objects: a scheme describes the mechanism of pagination at an abstract level, while a Link Object may express the concrete next-page navigation as a resolvable operation reference. Both may be used simultaneously.

11. Security Considerations

Pagination schemes expose information about API structure (parameter names, token formats) that is already present in the OpenAPI document. No additional sensitive information is introduced.

API authors SHOULD be aware that declaring totalCount in a response scheme may have privacy implications: exposing exact dataset sizes can leak information about the total number of records, even to unprivileged callers. This concern is not introduced by this specification but is worth noting when designing the API.

Cursor and page-token values MUST be treated as opaque by clients and MUST NOT be constructed or mutated by clients. Servers MUST validate token integrity and SHOULD sign or encrypt tokens to prevent enumeration attacks.

12. Change Log

Version	Date	Summary
0.1.0	Initial draft	Initial proposal: paginationSchemes component, three scheme types (pageNumber, pageToken, nextLink), auto-detection, operation-level application and overrides.

Appendix A: JSON Schema for This Extension

The following JSON Schema (Draft 2020-12) describes the structural constraints of all objects introduced by this extension. It is informative; Section 9 is normative.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://spec.openapis.org/extensions/pagination/0.1.0/schema.json",
  "title": "OpenAPI Pagination Extension Schema",

  "$defs": {

    "PaginationSchemesMap": {
      "type": "object",
      "additionalProperties": { "$ref": "#/$defs/PaginationSchemeObject" }
    },

    "PaginationSchemeObject": {
      "type": "object",
      "required": ["type"],
      "anyOf": [
        { "required": ["request"] },
        { "required": ["response"] }
```

```
      ],
      "properties": {
        "type": { "enum": ["pageNumber", "pageToken", "nextLink"] },
        "description": { "type": "string" },
        "autoDetect": {
          "oneOf": [
            { "type": "boolean" },
            { "$ref": "#/$defs/AutoDetectObject" }
          ]
        },
        "request":  { "$ref": "#/$defs/RequestPaginationFieldsObject" },
        "response": { "$ref": "#/$defs/ResponsePaginationFieldsObject" }
      },
      "patternProperties": { "^x-": {} }
    },

    "RequestPaginationFieldsObject": {
      "type": "object",
      "properties": {
        "queryParameters": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/RequestFieldObject" }
        },
        "bodyFields": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/RequestFieldObject" }
        },
        "headerFields": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/RequestFieldObject" }
        }
      },
      "patternProperties": { "^x-": {} }
    },

    "RequestFieldObject": {
      "type": "object",
      "properties": {
        "description": { "type": "string" },
        "role": {
          "enum": ["page", "pageSize", "offset", "pageToken", "cursor"]
        },
        "required": { "type": "boolean", "default": false },
        "schema": {}
      },
      "patternProperties": { "^x-": {} }
    },

    "ResponsePaginationFieldsObject": {
      "type": "object",
      "properties": {
        "bodyFields": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/ResponseFieldObject" }
        },
        "headers": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/ResponseFieldObject" }
        }
      },
      "patternProperties": { "^x-": {} }
```

```
    },

    "ResponseFieldObject": {
      "type": "object",
      "properties": {
        "description": { "type": "string" },
        "role": {
          "enum": [
            "nextPageToken", "nextCursor", "nextLink",
            "totalCount", "totalPages", "pageSize", "currentPage"
          ]
        },
        "schema": {}
      },
      "patternProperties": { "^x-": {} }
    },

    "AutoDetectObject": {
      "type": "object",
      "properties": {
        "matchQueryParams":     { "type": "boolean", "default": true },
        "matchBodyFields":      { "type": "boolean", "default": true },
        "matchResponseFields":  { "type": "boolean", "default": false },
        "matchHeaders":         { "type": "boolean", "default": false },
        "requireAll":           { "type": "boolean", "default": true }
      },
      "patternProperties": { "^x-": {} }
    },

    "PaginationApplicationObject": {
      "type": "object",
      "required": ["scheme"],
      "properties": {
        "scheme":      { "type": "string" },
        "overrides":   { "$ref": "#/$defs/PaginationSchemeObject" },
        "description": { "type": "string" }
      },
      "patternProperties": { "^x-": {} }
    }

  }
}
```

Appendix B: Comparison with securitySchemes

This appendix illustrates the structural parallel between the existing securitySchemes extension and this proposal, to assist readers already familiar with the authentication model.

Concept	securitySchemes	paginationSchemes
Component location	`components/securitySchemes`	`components/paginationSchemes`
Operation field	`security`	`pagination`

Concept	securitySchemes	paginationSchemes
Named definition	`Security Scheme Object`	`Pagination Scheme Object`
Strategy discriminator	`type (apiKey, http, oauth2, …)`	`type (pageNumber, pageToken, nextLink)`
Operation opt-out	`security: [] (empty array)`	`pagination: [] (empty array)`
Global default	`Not supported in OAS 3.x`	`Auto-detection provides a similar effect`
Multiple concurrent schemes	`Yes (array of requirement objects)`	`Yes (array of application objects)`
Inline definition	`Possible but not recommended`	`Not supported (must use components)`

Appendix C: Normative References

- **OAS 3.1** — OpenAPI Specification version 3.1.0. https://spec.openapis.org/oas/v3.1.0
- **RFC 2119** — Key words for use in RFCs to Indicate Requirement Levels. https://www.rfc-editor.org/rfc/rfc2119
- **RFC 8288** — Web Linking (Link header). https://www.rfc-editor.org/rfc/rfc8288
- **JSON Schema 2020-12** — https://json-schema.org/draft/2020-12/